Final Project Report

Ting Chia-Chen, Yuxin Ding, Yihe Ma, Kayla Tran

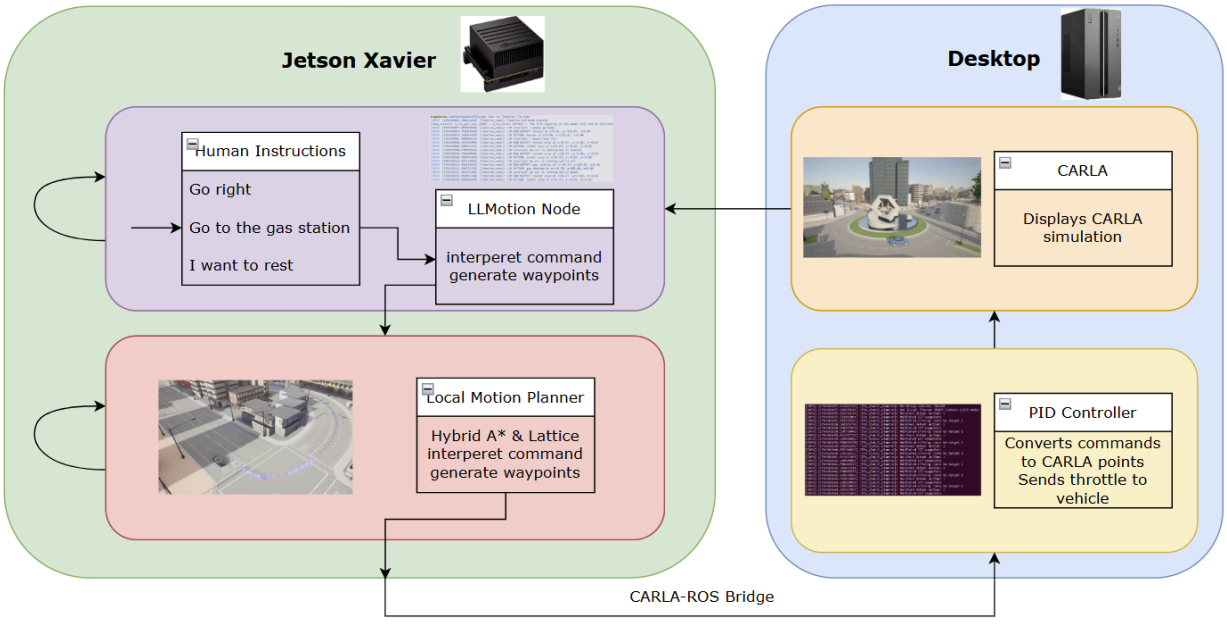
Project Overview / Background

1.1 Project Overview

Our project focuses on advancing autonomous driving through a Hardware-in-the-Loop (HIL) platform that connects real embedded hardware with a high-fidelity simulation environment. By linking a Jetson Xavier running ROS 2 with the CARLA simulator on a desktop workstation, we create a setup where planning and control algorithms can be executed on actual hardware while being evaluated safely in simulation. This allows us to study real-time behavior, communication performance, and system reliability without the risks of on-road testing.

As the project developed, we extended the system with support for multiple motion-planning approaches, including Hybrid A*, Lattice, and an instruction-driven module powered by large language models. This combination enables us to compare modern language-based reasoning with more traditional planners under the same HIL framework.

Through this work, we have successfully built a fully operational HIL platform that performs real-time trajectory execution, supports multi-planner evaluation, and maintains stable cross-device communication. The completed system provides a practical environment for testing autonomous-driving algorithms on real hardware while preserving safety, repeatability, and flexibility through simulation.



1.2 Background

Autonomous driving systems must be tested across a wide range of scenarios to understand how planning and control algorithms behave under real-time constraints. Doing this directly on a physical vehicle is costly and difficult to manage, and early-stage algorithms can pose safety concerns. A hardware-in-the-loop setup provides a practical alternative by running the core algorithms on real embedded hardware while relying on simulation to supply realistic environments, vehicle dynamics, and feedback.

This approach makes it possible to study timing behavior, communication delays, and overall system stability in a controlled and repeatable setting. At the same time, recent interest in instruction-driven autonomy has created a need for platforms that can evaluate both traditional planners and language-based reasoning side by side. These considerations motivated the development of LLMotion—a HIL platform that supports real-time planning, cross-device communication, and scenario testing without requiring on-road deployment.

Objectives

2.1 Overall objectives for project

- Develop a HIL digital-twin simulation platform that integrates CARLA, ROS2, and NVIDIA Jetson Xavier for real time autonomous driving experiments
- Enable closed loop testing where motion planning algorithms can generate and execute commands under real time vehicle simulation
- Integrate Talk2Drive as a LLM reasoning part for understanding human language instructions
- Evaluate the trade off between accuracy, computational feasibility, and real time performance between different local and cloud LLMs as well as our own motion planners

2.2 Original objectives for Fall Quarter

- Fix waypoint generation problem
- Complete traditional motion planner for vehicle control
- Integrate Talk2Drive, test latency for online model and local ones

2.3 Updated objectives for Fall Quarter

- Ensure the motion planner can generate correct trajectories and successfully drive the vehicle to the intended destination within CARLA.
- Tried Talk2Drive with models: GPT-4, GPT-4v, GPT-4-turbo, GPT-3.5-turbo, llama 2.70b, GPT 2 (local), deepseek r1(local), gpt4all (local), found out that multiple LLMs, including GPT-3.5-turbo, Llama 3.1, Llama 3.2, and Qwen works for the Jetson Xavier.
- Establish reliable DDS-based ROS2 communication between the Jetson Xavier and the Desktop to enable synchronized data exchange for HIL simulation
- Enable closed loop based navigation. With human natural language input, the system can

parse the instruction into a destination within CARLA, and the motion planner can autonomously generate and execute a trajectory that drives the vehicle to the correct location.

3. Setup Details for Project

3.1 Hardware

- Nvidia Jetson Xavier
    - Operating System: Ubuntu 20.04
    - Used as the embedded compute platform for the HIL setup
    - Runs ROS 2 Foxy, motion-planning algorithms (Hybrid A*, Lattice), DDS communication, and local LLM inference using Llama and Qwen models

- Desktop
    - Operating System: Ubuntu 22.04
    - Equipped with an NVIDIA GPU for simulation and visualization
    - Runs ROS 2 Humble, CARLA 0.9.15, the ROS–CARLA bridge, and the PID adapter used to execute trajectories within the simulator

- LLMs
    - Open AI Tokenizer using text tokens
    - GPT-3.5-turbo (online inference)
    - Llama 3.1 & 3.2 ((local inference on Jetson Xavier)
    - Qwen (local inference on Jetson Xavier)

3.2 Software

- Jetson Xavier Software Stack
    - ROS 2 Foxy
    - Python 3.8
    - Hybrid A* and Lattice planning modules
    - Local LLM inference engines (Llama 3.1, Llama 3.2, Qwen)
    - DDS communication
    - LLMotion instruction-processing components

- Desktop Software Stack
    - ROS 2 Humble
    - Python 3.10
    - CARLA 0.9.15 simulator
    - carla_ros_bridge for translation between CARLA and ROS topics
    - PID adapter for executing trajectories and maintaining the control loop

4. Standards Used/Considered

4.1 Software Revision Control

We are not using a VCS yet, artifacts are managed manually on both hosts. To reduce configuration drift and improve traceability, we will adopt Git (GitHub) next phase with a simple workflow: a protected main branch, short-lived feature branches, and pull requests for any planner/bridge changes. We will tag experiment baselines (e.g., carla-0.9.15_town04_pp_v1) and keep a lightweight CI to lint/build ROS workspaces. This will give us clear history, easier rollback, and reproducible releases for comparisons with TM/LLM results.

4.2 Communication Standards

Transmission Control Protocol (TCP) is being used to send control and command messages between CARLA and ROS2. DDS over ethernet is being used as well to support the communication of the CARLA-ROS bridge.

Prototypes

Figure 1

Caption: Baseline simulator bring-up with a Tesla Model 3 following waypoints in Town scene using CARLA 0.9.15 rendered via Vulkan on Ubuntu 22.04.

Details: This prototype verifies our graphics/runtime stack (Vulkan, asset loading, camera views) and confirms that deterministic waypoint playback works end-to-end. It serves as the ground truth baseline before we switch vehicle control from the built-in follower to our external planners running on the Jetson Xavier.

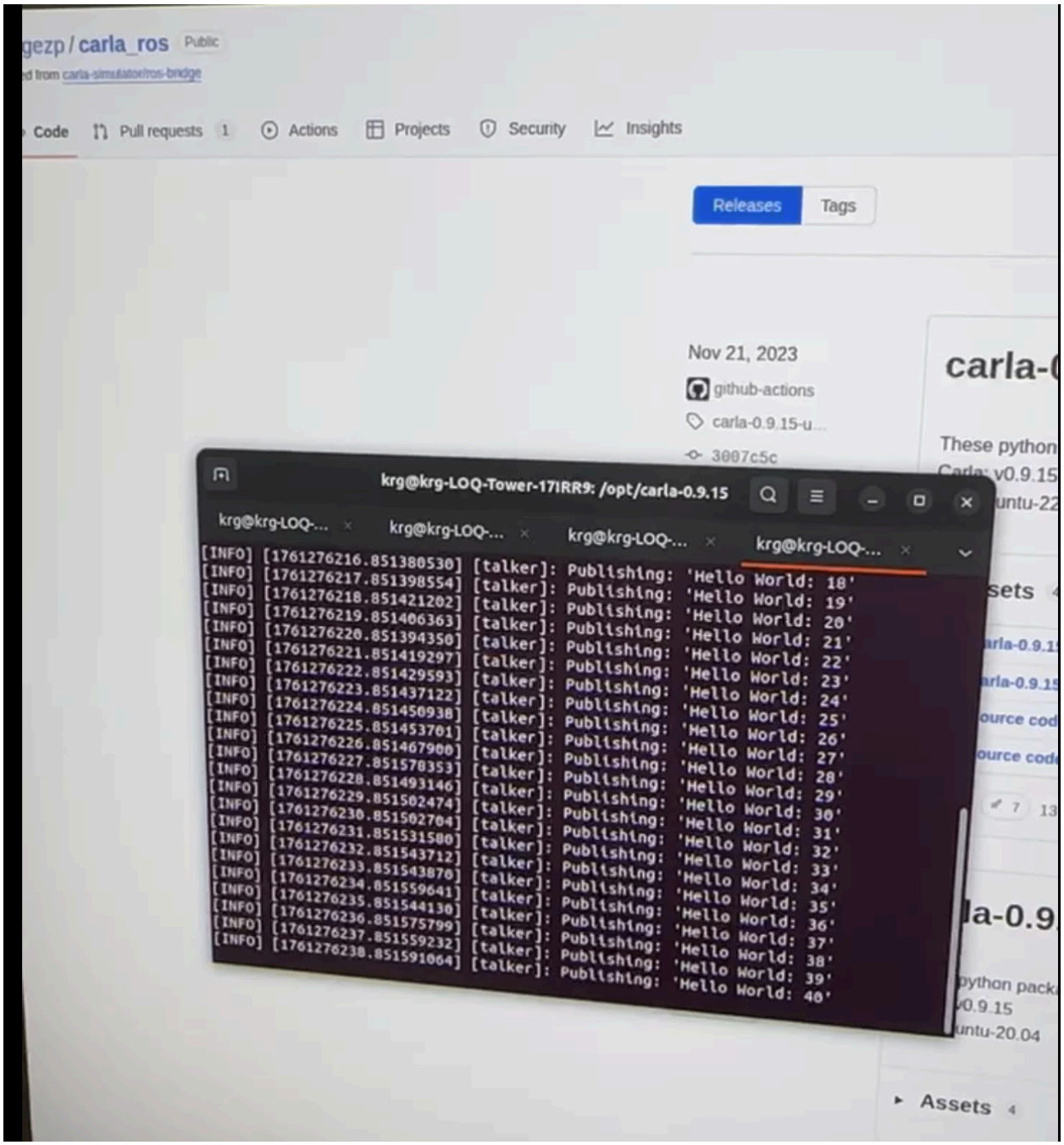


Figure 2

Caption: Humble talker example publishing in the shared DDS domain prior to using CARLA topics.

Details: This iteration verifies basic ROS 2 discovery and message flow on the desktop side independent of CARLA. Once confirmed, we swapped to /carla/* topics to ensure that type support and QoS match what carla_ros_bridge expects.

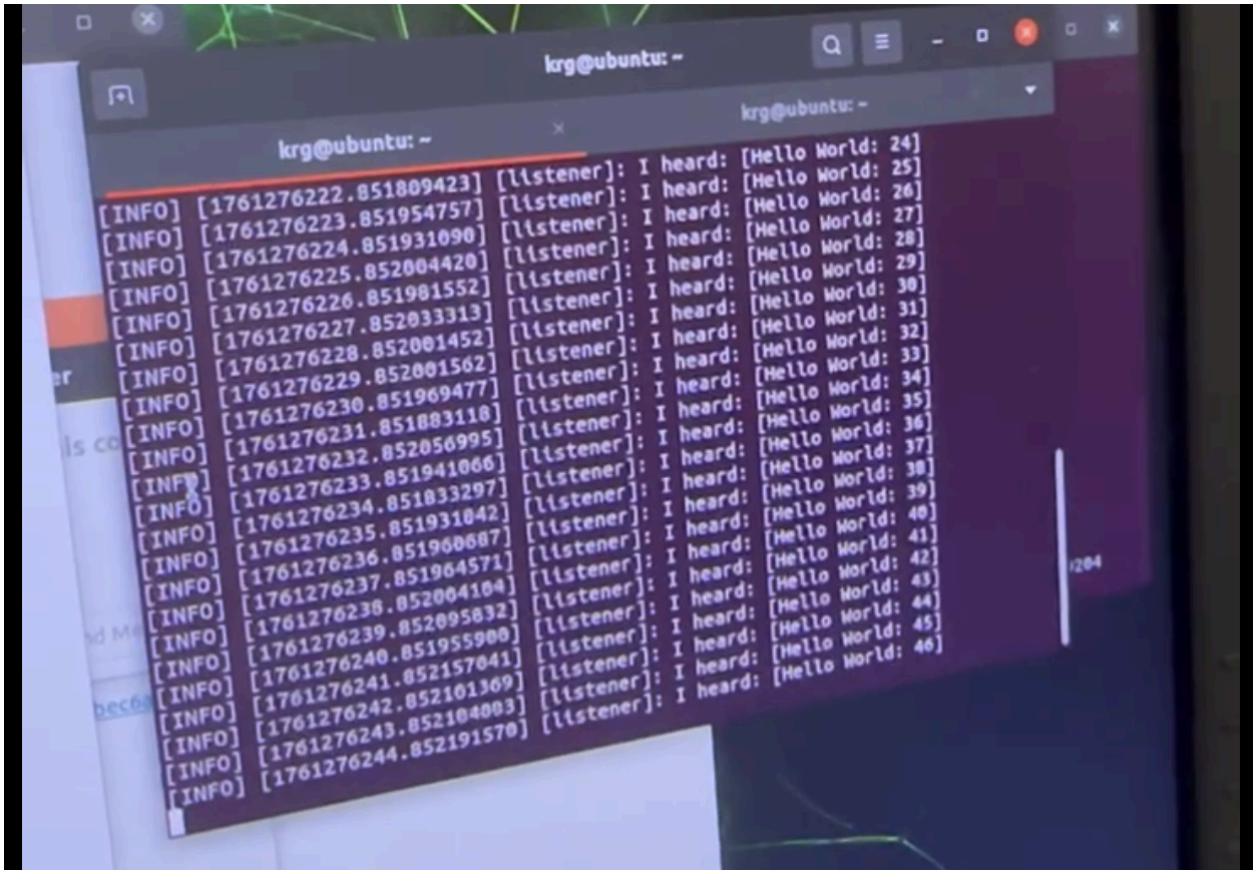


Figure 3

Caption: Xavier listener receiving the desktop talker messages across the LAN using Fast DDS and a shared ROS_DOMAIN_ID.

Details: This confirms Humble↔Foxy interoperability in our lab network and validates that Foxy can subscribe to desktop topics reliably. With this in place, we then subscribed to CARLA sensor streams (GNSS/IMU/RGB/LiDAR) and prepared the data path for local planners to publish back control commands.

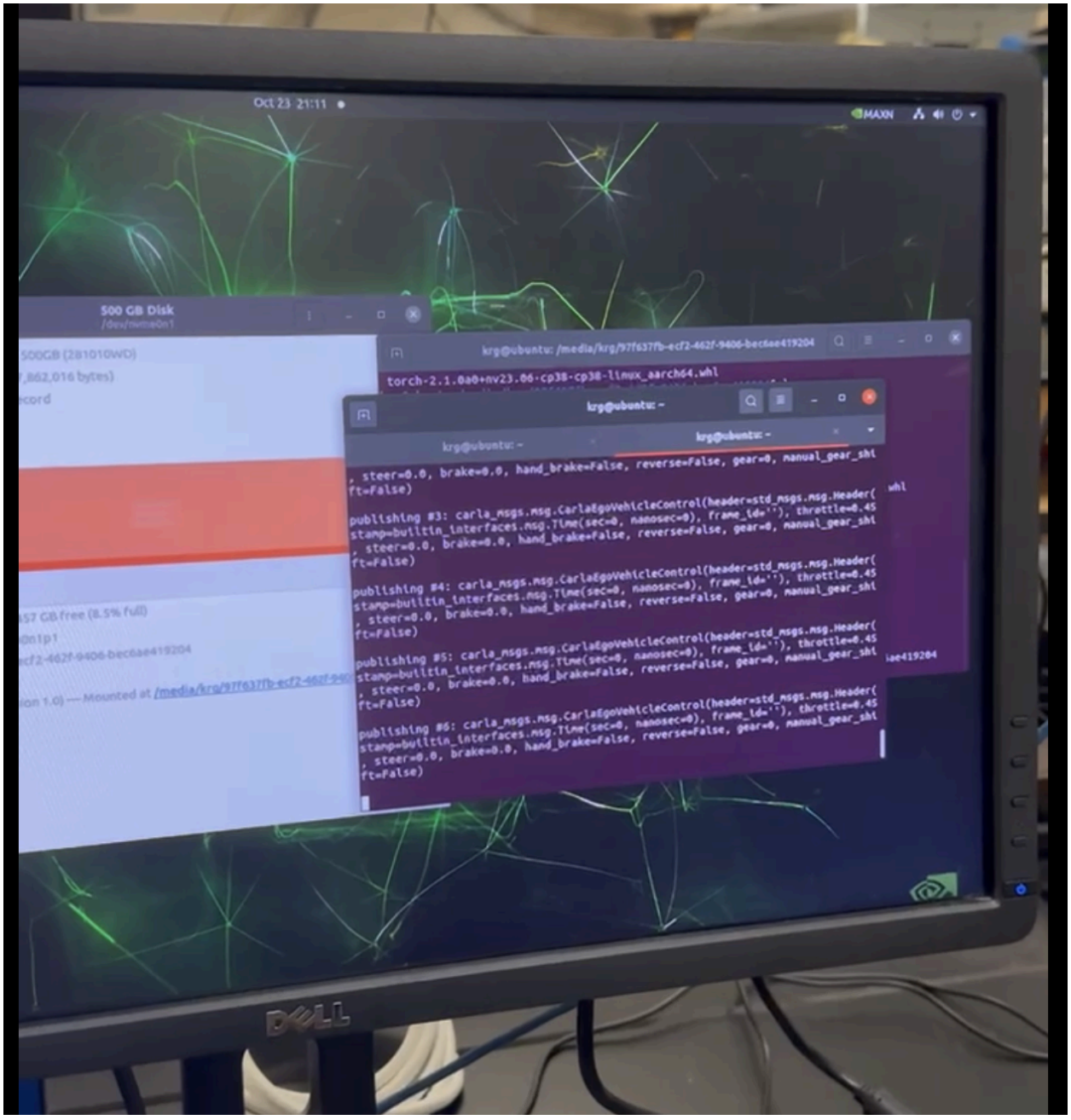


Figure 4

Caption: Xavier terminal continuously publishing CarlaEgoVehicleControl messages (throttle/steer/brake) toward the desktop host.

Details: This establishes the outbound leg of our Hardware-in-the-Loop path: Foxy node → Fast DDS → LAN → Humble host. We validated QoS and discovery using ROS_DOMAIN_ID=42, ROS_LOCALHOST_ONLY=0, and rmw_fastrtps_cpp. The goal of this iteration was to prove we can stream real-time control from the embedded side.

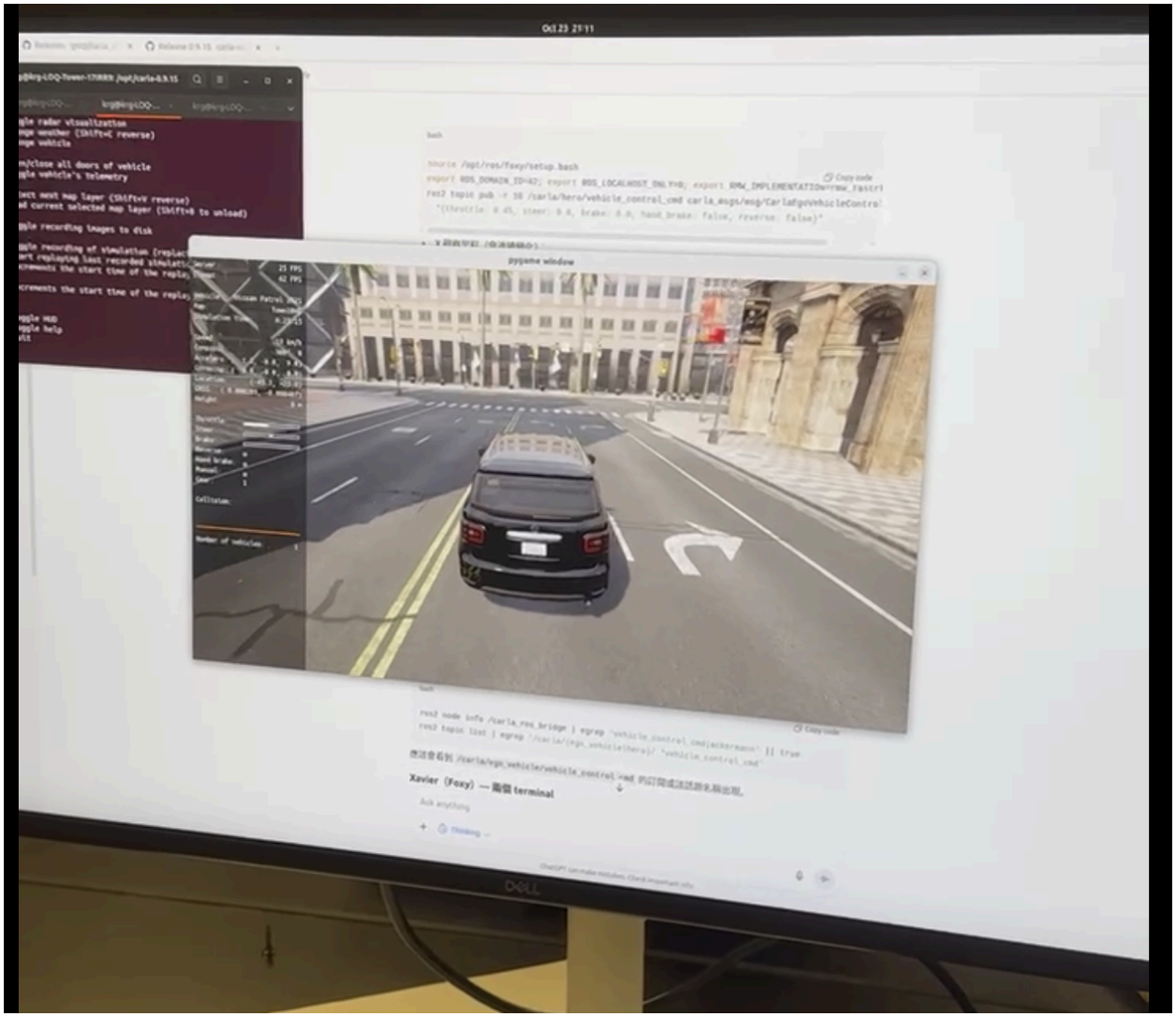


Figure 5

Caption: Humble host running carla_ros_bridge and manual_control successfully applies Xavier's commands; the pygame window shows the ego vehicle responding in real time.

Details: This closes the HIL loop (Xavier → Desktop → CARLA). With discovery stable across Humble↔Foxy, we confirmed command latency and smoothness are sufficient for low-frequency local planning. This iteration de-risks cross-version ROS 2 without Docker and readies the pipeline for planner outputs.

Talk2Drive

	Jetson	Jetson	Desktop CPU	Desktop CPU	Desktop CPU	Desktop	Desktop
Current ping to api.openai.com	69.03 ms	70.89 ms	414.30 ms	278.65 ms	371.43 ms	62.77 ms	55.14 ms
real	0m19.842s	0m21.907s	2m9.798s	1m17.674s	0m57.863s	0m35.089s	0m31.786s
user	0m13.605s	0m13.224s	0m34.595s	0m32.328s	0m31.408s	0m44.469s	0m45.482s
sys	0m1.871s	0m1.770s	0m34.902s	0m34.643s	0m34.545s	0m0.661s	0m0.543s

Figure 6

Caption: Measured round-trip network latency to api.openai.com and end-to-end runtime (real/user/sys) for Talk2Drive on both Jetson and Desktop.

Details: These measurements provide a reference for LLM-driven control responsiveness compared to local planners. We will use the same routes and speeds to A/B test LLM vs. classical planning, reporting tracking error, reaction time, and comfort metrics.

Experiments/Results so far

Test setup (hardware / software)

Desktop (Simulator Host): Ubuntu 22.04, Python 3.10.12, CARLA 0.9.15 (Vulkan), NVIDIA RTX 4060 (Driver 580.95.05), ROS 2 Humble, carla_ros_bridge (built, pcl_recorder skipped).

Jetson Xavier (Planner/Control): Ubuntu 20.04, ROS 2 Foxy.

Networking / DDS: Same L2 subnet, ROS_DOMAIN_ID=42, ROS_LOCALHOST_ONLY=0, RMW_IMPLEMENTATION=rmw_fastrtps_cpp (Fast-DDS).

Common: rolename standardized to ego_vehicle; CARLA topics via ros_bridge.

Test objectives

1. Verify stable CARLA 0.9.15 bring-up and topic publication.
2. Prove cross-version ROS 2 messaging (Humble ↔ Foxy) over Fast-DDS.
3. Close HIL loop: Jetson publishes control → Desktop ros_bridge → CARLA.
4. Record latency baselines for traditional control vs. LLM/Talk2Drive.
5. Test the accuracy of natural language translation to location commands.

Test cases

TC-1 — CARLA bring-up & waypoint baseline

Input: Launch CarlaUE4.sh (Vulkan); load Town; spawn one vehicle; enable waypoint follow.
Expected: Assets load; /carla/* topics available; vehicle follows lane waypoints smoothly.
Actual: Pass. Rendering stable; sensors and world topics visible; waypoint following OK.
Analysis: Validates simulator baseline for later external planners.

TC-2 — ROS 2 discovery across distros (Humble → Foxy)

Input: Desktop runs ros2 run demo_nodes_cpp talker; Jetson runs … listener; same Domain ID.
Expected: Jetson continuously receives “Hello World” messages.

Actual: Pass. Sustained reception confirmed.
Analysis: Confirms Humble–Foxy interop with Fast-DDS and current network settings.

TC-3 — ros_bridge ↔ CARLA 0.9.15 connection

Input: Start carla_ros_bridge.launch.py while CARLA is running; verify bridge logs and topics.
Expected: Bridge connects; ego topics appear (GNSS/IMU/cameras/control).
Actual: Pass after aligning the CARLA Python wheel to 0.9.15 (cp310).
Analysis: Version mismatch (0.9.13 vs 0.9.15) initially caused fatal error; resolved by installing the 0.9.15 wheel.

TC-4 — Jetson → Desktop vehicle control (closed loop)

Input: Jetson publishes carla_msgs/CarlaEgoVehicleControl to /carla/ego_vehicle/vehicle_control_cmd; Desktop shows pygame window.
Expected: Vehicle responds to throttle/steer/brake in real time.
Actual: Pass. Vehicle moves accordingly; motion visible in pygame and topics.
Analysis: Critical to standardize rolename = ego_vehicle; avoids "hero/ego" topic confusion.

TC-5 — Talk2Drive latency baseline

Input: Run Talk2Drive on Jetson and Desktop; record ping to api.openai.com and end-to-end time.
Expected: Desktop faster than Jetson due to stronger CPU/GPU/network.
Actual: Pass (matches expectation). Desktop shows lower network and wall-time; Jetson usable but slower.

Desktop Results:

LLM	gemma2:2b	mistral7b:instruct	qwen2.5-0.5b-instruct-q8_0	Llama-3.2-3B-Instruct-Q8_0.gguf	Llama-3.1-8B-Instruct-IQ4_XS	phi3:mini
Total end-to-end latency (s)	84.31	23.09	4.72	7.93	9.31	5.39
real (s)	98.78	25.56	8.28	12.58	13.23	7.92
user (s)	12.96	3.87	5.51	6.61	6.48	3.92
sys (s)	1.76	0.32	1.5	2.79	3.96	0.3

Jetson Xavier Results:

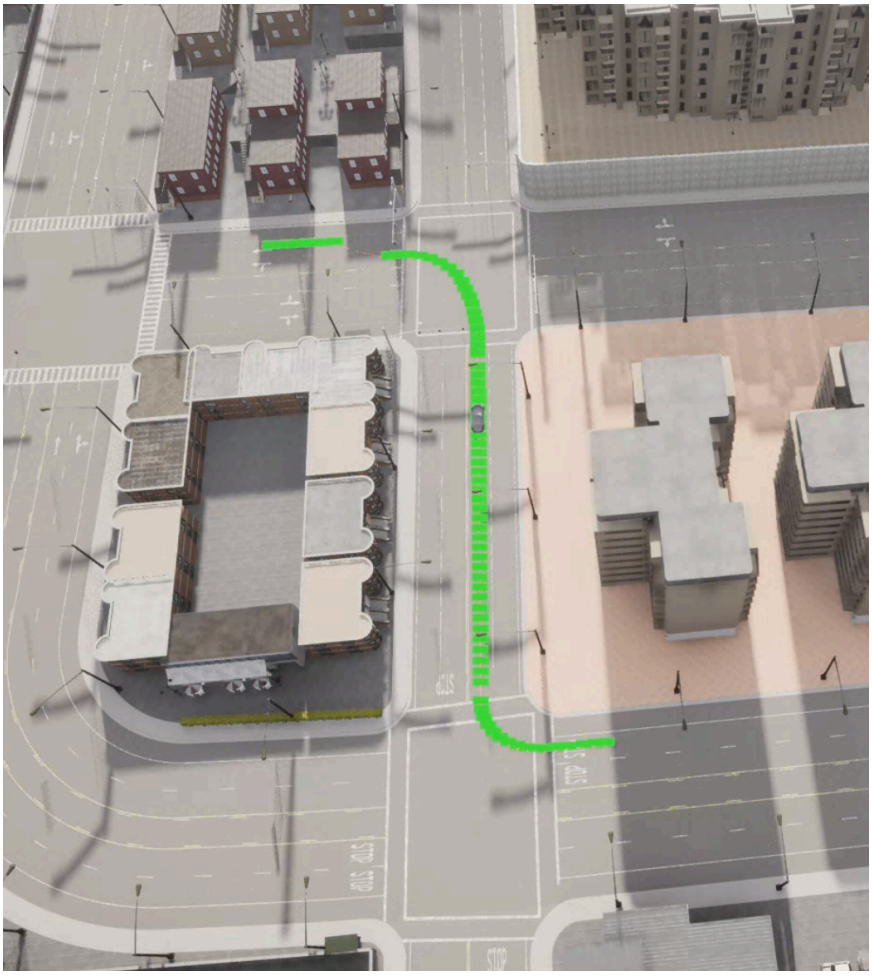
LLM	mistral:7b-instruct	phi3:mini	llama3:8b	qwen2.5-1.5b-instruct	gemma2:2b
Total end-to-end latency (s)	16.16	12.39	8.42	3.98	3.98
real (s)	18.62	14.81	10.88	6.47	6.47
user (s)	3.89	3.86	3.89	3.9	3.9
sys (s)	0.29	0.3	0.3	0.31	0.31

Analysis: Establishes LLM loop latency floor for later comparisons against local planners.

TC-6 — Instruction-to-Action End-to-End Behavior

Input: User types *"I'm not feeling well"* into the keyboard node on the Jetson Xavier.

Expected: LLMotion node correctly interprets the instruction as a request to go home, converts it into a coordinate which is defined as home in the map, and publishes the result to the /llm_action topic. The motion planner receives this destination, generates a planned path with visible waypoints in CARLA, and the vehicle follows the trajectory toward the home location.
Actual:



Analysis:

P-TC-7 — Instruction-to-Action End-to-End Validation (LLMotion Pipeline)

Input:
Execute the full Hardware-in-the-Loop setup, with CARLA launched on the desktop via ./run.sh and spawning the ego vehicle within a selected town environment. The desktop host runs the global planner to produce waypoints corresponding to the navigation goal.
On the Jetson Xavier, launch the following components in separate terminals:

1. keyboard_node — receives natural-language user instructions
2. llm_node — parses the instruction, determines the intended destination, and outputs corresponding target coordinates
3. follow_waypoint_node — processes global planner waypoints and generates motion references
4. pid_controller_node — applies actuation commands to /carla/ego_vehicle/vehicle_control_cmd

The user types a free-form instruction (e.g., “I want to go home”) into the keyboard node. The system interprets this text through LLM-based reasoning, maps it to a destination within CARLA, generates the associated global path, and publishes control signals to drive the simulated vehicle.

Expected Outcome:
The ego vehicle should begin moving within CARLA in a manner that reflects the semantic intent of the user’s natural language instruction. Green waypoint markers are expected to appear along the planned path, and the vehicle should attempt to follow these waypoints through observable steering, heading alignment, and trajectory progression.
Successful execution demonstrates full stack interoperability across all modules from language input and LLM interpretation through planner waypoint generation and PID-based actuation within the HIL environment.

Future Evaluation:
Once the end-to-end pipeline achieves stable and consistent execution, we plan to introduce quantitative evaluation metrics, including trajectory tracking accuracy, deviation from waypoints, control latency, and motion smoothness. These criteria will be finalized after robust waypoint-following performance has been achieved.

P-TC-8 — Local Planner Behavior Under LLM-Derived Navigation (Waypoint Tracking Assessment)

Input:
Under the same HIL configuration as P-TC-7, evaluate the behavior of our Jetson-executed local planner when following the global planner's waypoints generated from LLM-interpreted navigation instructions.
The local planner consists of our custom waypoint-follower implementation, which incorporates a Pure-Pursuit-style geometric heading rule, paired with a dedicated PID controller node responsible for generating vehicle control commands published to /carla/ego_vehicle/vehicle_control_cmd.
The CARLA vehicle is observed while attempting to execute the waypoint path that originates from the user's natural-language input.

Expected Outcome:
The vehicle should show clear intent to follow the provided waypoint sequence. Observable behaviors include overall progression along the plotted path, heading adjustments consistent with waypoint direction, and vehicle motion that correlates with the structure of the generated route. At this stage, deviations and imperfect tracking performance are expected, as the immediate goal is to confirm proper data flow, correctness of the control loop, and functional linkage between LLM-derived intent, trajectory generation, and actuation.

Future Evaluation:
Following successful stabilization of waypoint-tracking behavior, quantitative benchmarking will be introduced. Planned evaluation dimensions include, but are not limited to:

- lateral tracking deviation
- maximum departure from planned path
- control-loop execution rate
- actuation latency
- motion-smoothness indicators

Specific numerical thresholds have not yet been established and will be defined after the controller reliably maintains the intended waypoint trajectory in simulation.